

Paidly — Application Wiring Overview

Deep assessment · 26 August 2026 · Codebase + blueprint · Confidential internal briefing

Paidly is a **business operating system for SMEs** (South Africa—first: PayFast, ZAR). It is not an invoicing-only tool. Roughly **70–75%** of the intended SaaS architecture is production-wired. The commercial loop works end to end. Remaining value is concentrated in **document-model unification**, a first-class **Payment Intent** layer, and a short list of broken or stub surfaces.



Headline. Auth → clients/catalog → invoices/quotes/payslips → PDF/send → PayFast SaaS billing → admin-v2 is a live product. The highest-severity gaps are (1) recurring invoice edit 404, (2) sync-queue send that marks invoices sent without emailing, (3) dual document persistence, (4) missing `payment_intents` as system of record.

1. Contents

1. What Paidly is, and what it is becoming
2. Runtime architecture (browser, Supabase, API)
3. Blueprint systems vs shipped reality
4. Canonical data flow and entity layer
5. Session, realtime, and sync spine
6. Domain wiring deep-dive (18 surfaces)
7. Route map and admin consolidation
8. Scheduled jobs and dual-path debt
9. Improvement programme (P0–P3)
10. Recommended execution sequence

2. Product

Positioning in `docs/Paidly-Application-Blueprint.md`: shift from “invoice app” to **financial workflow platform**. Competitive edge is unification, not a single PDF:

Document → Payment Intent → Revenue System, plus shared Experience and Payment Intelligence.

Shipped (advanced) • Invoices, quotes, payslips (logical document engine, physical tables still split) • Clients, catalog/inventory, cash flow, reports • Public share tokens + client portal • SaaS billing v2 (plans catalog, trials, PayFast ITN, admin override) • Affiliates, waitlist, platform admin-v2	Blueprint remaining (~25–30%) • Canonical <code>payment_intents</code> (not yet a table) • Single persistence for document kinds • Observe stream used as Get Paid intelligence, not just logs • Experience consistency (PageTemplate + money CTA contract everywhere) • Multi-provider payment rails (PayFast still leaks into UI)
---	--

3. Runtime architecture

Three legs from the browser. They are not the same server.

LEG	CODE	TYPICAL USE	CONFIG
SPA	<code>src/</code> (Vite + React 18 + Router 7)	UI, TanStack Query, Zustand, EntityManager CRUD	<code>VITE_SUPABASE_*</code>
Supabase	Auth, Postgres + RLS, Realtime, Storage	System of record for tenants, documents, catalog, subscriptions	Project URL + anon / service role
Vercel <code>api/*.js</code>	Serverless catch-alls	Email, cron, PayFast, admin, portal, public shares	Same-origin <code>/api</code>
Express <code>server/</code>	<code>server/src/index.js</code>	Local/long-running API; rate limits apply only here	<code>VITE_SERVER_URL</code> when used

Deployment split. Express middleware and rate limits do *not* apply to traffic that only hits Vercel `api/*.js` . See `docs/API_DEPLOYMENT_MODEL.md` before diagnosing 429s.

3.1 What runs where (policy)

- **Entity CRUD** — browser → Supabase (RLS). Org scoped via `EntityManager` .
- **Privileged / secret work** — `/api` : email, ITN, admin, public tokens, cron, POS OAuth.
- **Never trust the SPA** for paid SaaS activation. PayFast ITN + DB is source of truth.

3.2 External services

SERVICE	ROLE	WIRING
Supabase	DB, Auth, RLS, Realtime, Storage	Fully
Vercel	Host, serverless, crons, redirects	Fully (Hobby cron budget)
PayFast	Tenant SaaS subscriptions + invoice once-off paths	Mostly (ITN verified; no Payment Intent abstraction)
Resend / SMTP	Transactional email	Mostly (live send + stub send coexist)
Anvil / react-pdf	Invoice/quote PDF generation	Fully
Yoco / Square POS	Sales ingress + optional stock decrement	Mostly (Settings + <code>api/pos</code>)

4. Blueprint systems vs shipped reality

SYSTEM	JOB	MATURITY	EVIDENCE
Identity	Who acts, for which org, with what authority	STRONG	Supabase Auth, <code>profiles</code> / <code>organizations</code> / <code>memberships</code> , RLS, <code>RequireAuth</code> , <code>ConnectionLifecycle</code> , <code>RuntimeCoordinator</code>
Document Engine	Issue, deliver, observe commercial documents	ADVANCED / DUAL	Code unification in <code>src/document-engine/</code> ; persistence still <code>invoices</code> / <code>quotes</code> / <code>payslips</code> plus hub <code>documents</code>
Payment Intent Layer	Canonical handoff document → rails	NOT BUILT	Specified in blueprint; no <code>payment_intents</code> table; PayFast logic still surface-adjacent
Relationship	Clients + catalog feeding compose	STRONG	Clients CRM, services/inventory, <code>adjust_inventory_stock</code>
Revenue & Ops	Capture, SaaS billing, cash, reports	STRONG BILLING	Plans catalog, ITN, trial cron, admin override; cash flow/reports read legacy tables
Payment Intelligence	Reminders, nudges, Get Paid funnel	PARTIAL	Client scheduler + cron stub; <code>document_events</code> exists for hub docs, not full invoice observe stream
Experience	One product feel (shell, money UX)	IMPROVING	<code>PageTemplate</code> , <code>CreateDocument</code> , still page-fork residue and dual send UX
Growth / platform	Affiliates, waitlist, admin-v2	MOSTLY	Apply/dashboard + admin moderation; Excel/Base44 leftovers remain

5. Canonical data flow

UI (pages / components)
→ Hooks (TanStack Query, local UI)
→ Services (src/services/* — orchestration, timeouts, gating)
→ Entity facades (@api/entities → EntityManager)
→ Supabase (+ RLS) | optional localStorage for unmapped entities
→ axios / fetch → /api/* (email, billing, admin, public tokens)

Policy: pages should not call `Invoice.list()` directly for list screens. Reference: `useInvoices` → `InvoiceListService` → `Invoice` entity → Supabase. Delivery (PDF/email/share) lives in `InvoiceService` / send APIs — a different concern from list reads.

5.1 Entity → table map (truth)

`getSupabaseTableForEntityName` in `src/api/entity/entityShared.js` is the gate. Unmapped names return `null` and fall back to local/broken persistence.

ENTITY	TABLE	STATUS
Client, Service, Invoice, Quote, Payment	matching plurals / <code>banking_details</code>	Mapped
RecurringInvoice	<code>recurring_invoices</code>	Mapped
Payroll / Payslip	<code>payslips</code>	Mapped
Expense, Task, Note	matching tables	Mapped
Supplier, PurchaseOrder (+ items)	<code>suppliers</code> , <code>purchase_orders</code>	Mapped
DocumentSend, MessageLog	<code>document_sends</code> , <code>message_logs</code>	Mapped
Vendor, Budget	<code>null</code>	UNMAPPED

6. Session, realtime, sync

This is one of the most heavily engineered parts of the product. Treat it as foundation, not a feature.

Session / connection

- `AuthContext.impl.jsx` — single auth owner; profile via `useAuth`
- `sessionRefreshScheduler` — coalesced refresh (visibility, online, heartbeat)
- `ConnectionLifecycleManager` — auth, realtime, wake, network signals
- `RuntimeCoordinator` — pauses HTTP while `AUTH_RECOVERING` / `RECONNECTING`
- `AppRecoveryLock` — sleep/wake: auth first, then realtime, then unlock UI

Live + offline

- Single multiplex Realtime channel: `paidlyRealtimeManager`
- JWT rotation: remove + recreate channel (no stale-auth join)
- `SyncEngine` + `useSyncQueueStore` for deferred writes
- `MutationCoordinator` + `client_operation_id` on invoices
- Logout purges persisted TanStack Query (IDB)

ID	RISK	STATUS
R-01	Direct <code>supabase.auth.getSession()</code> bypasses SessionCoordinator (~40 sites). High-risk: connectionHealth, affiliateClient, ActivityNotificationService.	Open, medium
R-02	Client-domain invalidation always cascades to invoices (correct for names; noisy for other edits).	Open, low
R-03	Broad <code>["invoices"]</code> query invalidation still present for legacy hooks.	Open, low

7. Domain wiring deep-dive

Score is completeness of UI → hook/service → API/DB → edge cases, not automated test coverage. Wiring legend: **FULLY** **MOSTLY**

PARTIAL **STUB / BROKEN**

7.1 Identity — Auth, signup, session **92**

Signup / Home → SupabaseAuthService + AuthContext → optional /api/auth/* → Auth + profiles + org bootstrap

- Invite / password / AcceptInvite routes live. Team RBAC: `CompanyContext` , `CompanyTeamService` , `/api/company` .
- Roles: `RequireAuth` , `RequireBusinessOwner` , `PERMISSIONS` for payroll/reports.
- **Gaps:** R-01 getSession bypasses; Excel signup side-path still imported (`ExcelUserService`).

7.2 Relationship — Clients **95**

Clients / ClientDetail / EditClient → useClients* → Client entity → clients

- Timeline: `ClientTimelineService` . Portal access fields exist. CSV mapping documented.

7.3 Relationship — Catalog & inventory **78–93**

/Services (Inventory.jsx) · /Inventory redirects here · EditCatalogItem · RPC adjust_inventory_stock

- Catalog CRUD and CSV import are live. Stock movements via shared RPC + `inventory_movements` .
- POS: `api/pos` , Square OAuth, Yoco connect, dashboard “POS sales today”.
- **Gap:** `ItemUsageService.js` still TODO (usage counts return 0; deletion-safety not real).

7.4 Document Engine — Invoices **82**

SUBFLOW	WIRING	PATH
Create	Fully	<code>/CreateInvoice</code> → <code>/CreateDocument/invoice</code>
List / edit / view	Fully	<code>useInvoices</code> , <code>EditInvoice</code> , <code>ViewInvoice</code> / <code>ViewDocument</code>
Payments (AR)	Fully	<code>usePaymentActions</code> → <code>Payment.create</code> ; inventory hooks on paid
PDF	Fully	Anvil / react-pdf; <code>/InvoicePDF</code> guest-safe with token
Interactive send	Mostly	<code>InvoiceActions</code> → Supabase edge <code>send-invoice-email</code> then fallback <code>POST /api/send-invoice</code>
Sync-queue send	Stub	<code>syncJobProcessor</code> → <code>sendInvoiceToClient</code> marks status sent only (TODO comment)

P0 — silent false send. `InvoiceSendService.sendInvoiceToClient` updates `status: 'sent'` and records a document send, then comments `TODO: Implement actual email sending via API` . Offline/sync `SEND_INVOICE` jobs use this function. The interactive UI path emails; the queue path may not.

7.5 Document Engine — Recurring invoices **55**

- List, create, pause/resume: wired to `recurring_invoices` .
- Generation: `RecurringInvoiceService.checkAndGenerateDueInvoices()` runs in the browser when the list page loads. **No Vercel cron** for this. Invoices only generate if an owner opens the app.
- **Broken edit:** `src/pages/EditRecurringInvoice.jsx` exists. It is *not* registered in `src/pages/index.jsx` . List navigates with `createPageUrl("edit-recurring-invoice")` → `/Edit-recurring-invoice` (404 / NotFound).

7.6 Document Engine — Quotes **90**

Quotes / CreateQuote / EditQuote / QuotePDF / PublicQuote · convert-to-invoice in QuoteActions

- Send helpers share `InvoiceSendService` tracking pixels. Public token loaders: `api/_publicQuoteShared.js` .
- Quote → invoice conversion is implemented (dedicated path + `source_quote_id`), not just marketing copy.

7.7 Document Engine — Payslips **88**

Payslips CRUD + PDF + PublicPayslip · RBAC `MANAGE_PAYROLL` / `VIEW_OWN_PAYSLIPS`

- Still a separate table. Hub also has leave / expense-claim typed documents with different depth.

7.8 Documents hub 72

Documents.jsx / DocumentDetail → DocumentService → public.documents + document_items + document_events

- Migrations: 20260418220000_documents_engine_core.sql , hub backfill 20260603120000_documents_business_hub.sql .
- Leave request and expense claim create routes exist. Import marked coming soon.
- **Structural risk:** production invoices/quotes/payslips still live in legacy tables. Hub is additive. Dual-write can diverge.

7.9 Revenue — SaaS subscriptions & trials 91

This is one of the most complete subsystems.

- Plans catalog in DB (Starter R50 / Business R150 / Growth R350 + annual). SPA does not set price.
- POST /api/subscriptions/create|change|cancel ; status from verified PayFast ITN only.
- 7-day server-side trial for org owners created on/after 2026-08-20 UTC. Invited members do not get a trial.
- Admin PATCH can extend/grant/suspend/cancel and sets admin_override so expiry jobs cannot revert it. Migration 20260820200000_trial_lifecycle_admin_override.sql .
- Legacy POST /api/payfast/subscription returns 410.
- Reporting counts only payment_history.payment_status = completed after cutoff — not trials or admin grants.

7.10 Revenue — Expenses, cash flow, accounting, budgets 80 / STUB BUDGETS

- Expenses persist via CashFlow + ExpenseForm → expenses . Used by Reports and AccountingDashboard.
- Accounting is FeatureGate-wrapped; not a first-class nav product.
- **Budgets** page exists; Budget entity is not in the Supabase table map → local/broken.
- **Vendors** admin page similarly unmapped; overlap with live suppliers + purchase orders.

7.11 Experience — Portal, public shares, settings 90

- Public: /PublicInvoice , /view/:token , quote/payslip public routes; api/public-share.js .
- Portal: ClientPortal.jsx → clientPortalClient → /api/client-portal (login, data, payment, messages).
- User Settings: profile, logo storage, banking, templates, reminders tab, team, POS.
- Day-to-day branding is profile logo + document_brand_* , not full multi-company UI.

7.12 Multi-brand / companies 45

- Schema: companies , invoices.company_id , attachInvoiceCompany in entityShared.
- Team/workspace is more complete than brand picker. Product UI for managing brands / choosing company_id on create is thin.
- **Recommendation:** ship the picker or stop claiming multi-brand in positioning.

7.13 Growth — Affiliates 80

Landing / apply / dashboard · affiliateClient · server affiliate admin · api/affiliates · referral on signup

7.14 Platform — Admin-v2 84

Single UI shell. Dozens of legacy /Admin* and /admin/* routes redirect into:

ROUTE	ROLES	BACKING
/admin-v2	admin, management, sales, support	Dashboard KPIs via admin APIs
/admin-v2/users	same	api/admin platform users
/admin-v2/messages	same	Broadcast + platform user messages
/admin-v2/subscriptions	admin, management, sales	Plans, ITN reporting, PATCH override
/admin-v2/affiliates	admin, management, support	Application moderation
/admin-v2/waitlist	same as dashboard	Waitlist table
/admin-v2/settings	admin, management	admin_settings via API (not localStorage)
/admin-v2/audit-log	admin, management	Hybrid local/server

Dead AdminDataService chain removed. Some docs (reality matrix) are stale vs code. System-health probes remain soft.

7.15 Email, tracking, reminders 75

PATH	STATUS
Interactive invoice/quote email (edge + <code>/api/send-invoice</code>)	Wired
Open / link tracking (<code>/api/email-track</code> , <code>_emailTrackShared.js</code>)	Wired
In-app ActivityNotificationService	Wired (direct getSession)
Admin broadcast email cron (10:00 UTC)	Wired
Browser PaymentReminderScheduler	Best-effort while app is open
<code>/api/cron/payment-reminders</code>	Handler returns <code>skipped: not_scheduled</code> — explicitly waiting on Payment Intelligence Wave 4. Not in vercel.json crons.
Sync-queue sendInvoiceToClient	Status-only stub

8. Scorecard (all assessed domains)

DOMAIN	SYSTEM	STATUS	SCORE	PRIMARY GAP
Clients	Relationship	FULLY	95	—
Catalog	Relationship	FULLY	93	ItemUsageService stub
Auth / session	Identity	FULLY	92	getSession bypasses
SaaS billing / trials	Revenue	FULLY	91	—
Quotes	Document	FULLY	90	—
Portal / public shares	Experience	FULLY	90	—
User settings / branding	Experience	FULLY	90	Not multi-brand
Payslips	Document	FULLY	88	Separate table vs hub
Admin-v2	Platform	MOSTLY	84	Health / stale docs
Invoices (create/pay/PDF)	Document	MOSTLY	82	Dual send + dual store
Expenses / cash flow	Revenue	MOSTLY	80	Not a product nav item
Affiliates	Growth	MOSTLY	80	—
Inventory / stock	Relationship	MOSTLY	78	Usage analytics stub
Email / reminders	Intelligence	MOSTLY	75	Cron skipped; queue stub
Documents hub	Document	MOSTLY	72	Parallel model
Recurring invoices	Document	PARTIAL	55	Edit 404; client-only gen
Multi-brand	Identity	PARTIAL	45	Schema without UX
Vendors / budgets	Revenue	STUB	25	No table map

9. Dual-path debt

These are the structural reasons the product can feel “wired twice” or disagree with itself.

AREA	PATH A (LIVE)	PATH B (PARALLEL)	RISK
Documents	invoices / quotes / payslips tables	public.documents hub	Double truth; reports vs hub lists diverge
Invoice send	Edge function + POST /api/send-invoice	InvoiceSendService TODO used by sync queue	Marked sent without email
API host	Vercel api/*.js	Express server/	Rate-limit / middleware mismatch
Reminders	Browser scheduler	Cron endpoint disabled on purpose	Reminders only if user is online
Recurring generation	On RecurringInvoices page load	No cron job	Missed cycles when owners are idle
Admin docs	admin_settings in DB	Reality-matrix still mentions localStorage / deleted services	Onboarding confusion

10. Scheduled jobs (Vercel)

Hobby plan uses a single `/api/cron?job=` dispatcher. Auth: `Authorization: Bearer CRON_SECRET`.

JOB	SCHEDULE (UTC)	IN VERCEL.JSON CRONS?	BEHAVIOUR
expire-trials	07:00	Yes	Expire trials; respect admin_override
subscription-dunning	08:30	Yes	Tenant billing dunning
subscription-prebill-invoices	09:00	Yes	Pre-bill SaaS invoices
admin-broadcast-email	10:00	Yes	Queued platform broadcasts
payment-reminders	—	Rewrite only, not scheduled	Returns skipped until Wave 4
recurring invoice generation	—	No	Client-only

11. Improvement programme

P0 — User-visible correctness

	ITEM	EVIDENCE	FIX
P0	Register recurring edit route	Page file exists; missing from index.jsx; createPageUrl capitalizes only first letter → /Edit-recurring-invoice	Add /EditRecurringInvoice (+ aliases); change navigate to createPageUrl("EditRecurringInvoice")
P0	Unify invoice send	sendInvoiceToClient TODO; syncJobProcessor SEND_INVOICE	Call the same path as InvoiceActions (/api/send-invoice or edge function); never mark sent without provider success

P1 — Architecture / product truth

	ITEM	WHY IT MATTERS
P1	Choose a documents system of record	Either migrate invoice/quote/payslip reads to documents, or fence hub types (leave, claims, contracts) so dual-write cannot diverge. Blueprint still lists single persistence as roadmap.
P1	Add payment_intents	Blueprint v1 contract: intent_id, document_ref, amount/currency snapshot, rail, status. Without it, PayFast leaks, multi-rail is hard, sticky payment CTAs stay page-specific.
P1	Server cron for recurring generation	Same pattern as expire-trials. Owners should not need to open the app for cycles to fire.
P1	Multi-brand UX or retire the claim	Schema is ready; compose picker is not. Positioning currently oversells this.
P1	Vendors / Budgets persistence	Map to Supabase or remove routes. Align Vendors with suppliers to avoid two supplier concepts.
P1	Expand document_events on legacy invoices	Hub already inserts events. Invoice observe (sent/opened/clicked/paid/reminded) should be one stream for Get Paid.

P2 — Hardening

- P2 Migrate bursty getSession callers to getStableSession (R-01).
- P2 Finish queryPolicies keys; drop bare ["invoices"] invalidation (R-03).
- P2 Enable or delete payment-reminders cron; stop documenting it as scheduled if it is not.
- P2 Admin system-health real probes; refresh stale admin markdown.
- P2 Split AuthContext stable vs volatile to cut re-renders on token refresh.
- P2 Sync queue max-size + failed-job TTL; org_id on jobs for future multi-org replay.

P3 — Cleanup

- P3 Remove Excel/Base44 leftovers from shell/signup/dashboard.
- P3 Implement or delete ItemUsageService TODOs.
- P3 Documents Import; debounce Zustand persist writes; move large collections off localStorage to IDB.
- P3 Roll PageTemplate remaining list pages; Experience checklist for new screens.

12. Blueprint v1 contract (still open)

From the Application Blueprint “immediate execution order” — none of these five are fully done:

1. Add payment_intents table.
2. Integrate intents into document lifecycle (Deliver → Observe → Intent → Settle).
3. Expand document_events coverage and ingestion (especially for invoices, not only hub docs).
4. Sticky right panel CTAs driven by intent status (Pay now / Retry / Send reminder) — one contract.
5. Event- and due-date-driven reminders as a real scheduled job (currently skipped).

Acceptance: invoice settlement only from provider-verified events; no “mark paid” shortcuts that skip the intent.

13. Recommended sequence

1. **Week 0 — P0 fixes.** Recurring edit route + send-path unification. User-visible correctness; small PR surface.
2. **Then — documents SoR decision.** Written ADR: hub-for-new-types-only vs migrate commercial docs. Do not keep growing both.
3. **Then — payment_intents + webhook reconciliation.** Wrap PayFast; keep payments as settlement records.
4. **Then — intelligence crons.** Recurring generation + payment-reminders with reminder_settings in DB.
5. **Then — product honesty.** Multi-brand UI or drop; Vendors/Budgets map or retire.
6. **Ongoing — runtime hygiene.** R-01 / R-03, AuthContext split, Excel leftover removal.

What not to do next. Do not add another document kind, another payment provider, or another admin page until P0 send/recurring bugs are closed and a documents SoR is chosen. The architecture is already ~70% there; more surface area without those decisions increases dual-path debt.

14. Key file index

CONCERN	START HERE
Product strategy	<code>docs/Paidly-Application-Blueprint.md</code>
Connection map	<code>docs/HOW_EVERYTHING_CONNECTS.md</code>
Routes	<code>src/pages/index.jsx</code>
Entities	<code>src/api/entity/EntityManager.js</code> , <code>entityShared.js</code>
Document types	<code>src/document-engine/</code>
Send (stub vs live)	<code>src/services/InvoiceSendService.js</code> , <code>InvoiceActions.jsx</code> , <code>server/src/index.js</code>
Sync queue	<code>src/lib/syncJobProcessor.js</code>
Runtime	<code>src/core/runtime/RuntimeCoordinator.ts</code>
Billing	<code>api/subscriptions/</code> , <code>api/payfast-handler.js</code> , <code>docs/SUBSCRIPTION_BILLING_SCHEMA.md</code>
Cron	<code>api/cron.js</code> , <code>vercel.json</code>
Hub documents	<code>src/services/DocumentService.js</code>

Generated 26 August 2026 from the Paidly repository (SPA + api/ + server/ + supabase migrations + blueprint). Scores are engineering judgment of wiring completeness, not automated coverage. Companion interactive canvas: `paidly-app-overview.canvas.tsx`. HTML source: `docs/Paidly-Application-Wiring-Overview.html` · regenerate with `scripts/generate-wiring-overview-pdf.mjs`.